Learnova: Faithful Reconstruction of Lecture Slide Decks with an Explainable Visual Decision Engine, a Cognitive-Load Fitness Metric and Load-Optimal Segmentation

Swayam Devendra Panchal*, Kawaljeet Singh Bharaj*, Jeevesh Bharat Bodhani*,
Rishit Pankajkumar Lahoti* and Pratik Kanani†
*Student, †Head of Department
Department of Artificial Intelligence and Data Science, Dwarkadas J. Sanghvi College of Engineering, Mumbai, India
panchalswayam35@gmail.com

***Abstract*—Teachers rarely build a new slide deck for every lecture; they reuse old ones, and those decks are often hard to learn from. Presentation generators based on large language models create new decks from a prompt, but they paraphrase the source, drop list items and cannot be checked. We present Learnova, a system that rebuilds an existing deck instead of generating a new one and keeps every point of the source. This paper describes the methods we designed for it: (i) geometric rules that recover the structure of a deck exported to PDF, including a dynamic program for titles that lost their spaces and a graphic-ratio test that decides when optical character recognition may replace a picture; (ii) an explainable Content Decision Engine that maps each slide to one of more than twenty visual families with a confidence and a reason; (iii) Pedagogical Slide Fitness (PSF), a slide-quality metric of the form $E^{\alpha}(1-L)^{\beta}C^{\gamma}$ derived from cognitive-load and multimedia-learning theory; (iv) CLASS, a dynamic program that splits an overloaded slide into consecutive slides of minimum total load; (v) a glyph-measured layout method that guarantees no word is broken across lines; and (vi) an illustration stage in which diagrams written by a language model are sanitised and validated, with offline fallbacks and a scheduler for a pool of API keys. On 101 labelled slides the decision engine reached 96.8–97.4% accuracy at under 1 ms per slide; on six real decks (28–216 slides) the system kept 98–100% of the source facts, removed every untitled slide and produced no overflowing or broken text; CLASS never did worse than even splitting and was strictly better on 37% of overloaded sections.**

***Index Terms*—presentation generation, document structure recovery, explainable AI, cognitive load, slide segmentation, layout, educational technology**

I. Introduction

The slides a student revises from are usually the slides the teacher has used for several years. In decks we collected from our own semester we kept seeing the same problems: titles missing or reduced to "Slide 9", long lists of bullets, equations and diagrams pasted as pictures, and nothing a student could interact with. Redesigning such a deck by hand takes hours, so in practice it is not done.

AI presentation tools [1]–[4] address a different task. They produce an attractive deck from a prompt or a pasted document, but they rewrite the content as they go. Summarisation research has shown why this is risky: abstractive models omit and invent content [5], [6]. For a study aid that is not acceptable, because the student cannot see what was lost.

Our position is that the source content should be treated as fixed and only its presentation should change. Every structural decision—what a slide says, which visual shows it, how much goes on one slide—is made by an explicit rule whose reason can be shown, and a language model is used only where its output can be checked against the slide. The contributions of this paper are the methods behind that position:

1) rules for recovering the structure of slide PDFs, including title segmentation by dynamic programming and an OCR acceptance test based on a graphic ratio (Section IV);
2) an explainable Content Decision Engine (Section V);
3) the PSF metric and the CLASS segmentation algorithm (Section VI);
4) glyph-measured text fitting with a no-broken-word guarantee (Section VII);
5) sanitised AI diagrams with offline fallbacks and a key-pool scheduler (Section VIII);
6) an evaluation on gold sets and real decks, reproducible from the project repository (Section IX).

II. Related Work

Table I compares Learnova with representative systems. PPSGen [7] learned sentence salience and selected sentences for slides; DOC2PPT [8] and D2S [9] generate slides from scientific papers with neural summarisation. These systems take a paper as input and produce shorter text; none starts from an existing deck. Automatic visualisation design—APT [10] and Draco [11]—encodes design knowledge as rules or constraints; our decision engine applies the same idea to sentences. The assertion–evidence approach [12], [13], Mayer's principles [14] and cognitive load theory [15], [16] give the design targets that PSF formalises. CLASS follows the structure of Knuth–Plass line breaking [17], with slides in place of lines and cognitive load in place of badness.

TABLE I
CAPABILITIES OF RELATED SYSTEMS

System	Input deck	Keeps content	Explains choices	Quality metric	Off-line	Inter-active
PPSGen [7]	–	–	–	–	✓	–
DOC2PPT [8]	–	–	–	–	✓	–
D2S [9]	–	–	–	–	✓	–
Gamma, Copilot, Canva	part.	–	–	–	–	–
Learnova	✓	✓	✓	✓	✓	✓

III. System Overview

Fig. 1 shows the pipeline. An upload (PPTX, slide PDF, Word document or typed text) is converted into a Markdown intermediate representation with per-slide metadata. A planner splits overloaded slides with CLASS and adds an agenda, checkpoint questions and a glossary. The decision engine assigns a visual to each slide, the illustration stage attaches a picture, and two builders write an editable PowerPoint file and a web deck. A layout audit and a fact check run on the finished files. Stages that use a language model fall back to rules when no key is available.

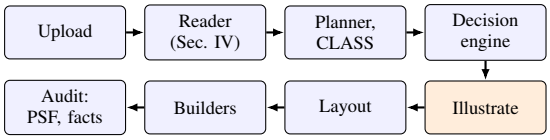


Fig. 1. Learnova pipeline. The orange stage uses a language model when a key is available and offline rules otherwise.

IV. Recovering Slide Structure

The reader works on text lines, each with a box (x_0, y_0, x_1, y_1), a font size s and a font. Table II lists the thresholds; they were set on development decks and kept fixed for the evaluation.

Furniture. Let n be the number of pages. A line in the top 12% or bottom 14% of a page is furniture if its normalised text appears in that band on at least

$$m(n) = \max\big(3,\ \lfloor 0.3\,n \rfloor\big) \tag{1}$$

pages. Pictures are treated in the same way but counted only where they are drawn: some exporters list every image of a deck in one shared resource dictionary, and counting resources would mark every figure as a logo.

Titles. With body size $\bar{s}$ (the size carrying the most characters), the largest line ℓ^* in the top third of a page of height H is the title if

$$s(\ell^*) \geq 1.18\,\bar{s} \quad \vee \quad \big(\mathrm{bold}(\ell^*) \wedge y_0(\ell^*) < 0.2H\big). \tag{2}$$

A following line continues the title if its size differs by less than 0.8 pt, it starts within 60 pt horizontally and its vertical gap g satisfies $-0.5\,s < g < 0.9\,s$ (boxes from some exporters overlap, hence the negative bound).

Levels. Bullet-glyph positions are clustered with tolerance $\tau = 9$ pt into anchors $a_1 < \cdots < a_K$, and an item starting at x has level

$$\ell(x) = \arg\min_k |x - a_k|. \tag{3}$$

Glued titles. For a capitalised token t of length L_t that is not a word and a vocabulary V built from the deck and a dictionary,

$$B(j) = \max_{\substack{i<j,\ j-i\leq 18 \\ t_{i:j}\in V}} \big(B(i) + (j-i)^2\big), \qquad B(0) = 0, \tag{4}$$

and the segmentation behind $B(L_t)$ is used when it has at least two words. Squaring the word length favours a few long words over many short ones.

Picture-only slides. If the largest picture covers at least 45% of the page and the page has at most 12 words, the picture is read with Tesseract [18]. With I the dark pixels and M the padded word boxes, the graphic ratio is

$$\gamma = \frac{|I \setminus M|}{|I|}. \tag{5}$$

The OCR text replaces the picture only if the mean confidence $\bar{c} \geq 62$, there are at least 12 words, $\gamma \leq 0.42$, at most 15% of lines are symbol debris and the tokens do not look like a photographed table; otherwise the picture is kept as a figure.

Chart debris and PPTX input. If a slide's items average fewer than four words and at least half of their tokens contain digits, they are the ticks of a drawn chart, and the region below the title is rendered as an image instead. A PowerPoint file is first rendered to PDF by LibreOffice (cached by content hash) and read by the same rules. Algorithm 1 summarises the procedure.

TABLE II
READER THRESHOLDS (FIXED FOR ALL EXPERIMENTS)

Rule	Symbol	Value
Furniture bands (top / bottom)	–	12% / 14%
Furniture repetition	$m(n)$	$\max(3, 0.3n)$
Title size ratio	–	1.18
Level clustering tolerance	τ	9 pt
Longest glued word	–	18
OCR trigger: area / words	–	0.45 / 12
OCR acceptance: confidence	$\bar{c}$	62
OCR acceptance: graphic ratio	γ	0.42
OCR acceptance: debris lines	–	15%

Algorithm 1 Reading one page of a slide PDF

$\mathcal{L} \leftarrow$ text lines; drop furniture by (1) and page counters
merge words on one baseline; split glued titles by (4)
$F \leftarrow$ figures drawn on the page (not shared resources)
if F_1 covers $\geq 45\%$ and the page has ≤ 12 words **then**
    run OCR; **if** accepted **then** $\mathcal{L} \leftarrow$ OCR lines **else** keep F_1
end if
title $\leftarrow$ largest top line, wrapped lines rejoined
items $\leftarrow$ bullets with levels $\ell(x)$, wrapped lines rejoined
if items are chart debris **then**
    figure $\leftarrow$ region below the title; items $\leftarrow \varnothing$
end if
return (title, items, tables, figure, kind)

V. Explainable Content Decision Engine

For each slide a set $\mathcal{D}$ of 27 detectors is run. A detector d returns nothing or a tuple $(f_d, v_d, c_d, D_d, r_d)$: a family, a variant, a confidence $c_d \in [0,1]$, the data the renderer needs and a reason in plain words. Tables, section outlines and reference lists are tested first because they are unambiguous. The remaining candidates $\mathcal{C}$ pass through exclusion rules X (a cycle removes a linear process; a definition removes statistic tiles; a timeline removes cause–effect), and

$$d^* = \arg\max_{d \in X(\mathcal{C})} c_d, \qquad \hat{c} = \min(c_{d^*},\, 0.97). \tag{6}$$

The next three candidates are kept as alternatives for the editor. If $\mathcal{C}$ is empty, the slide becomes key-point cards that carry every sentence in full, so choosing a visual never removes text. Card headings are made from the sentence by removing discourse cues and pronouns, keeping the first three content words, cutting at an unbalanced parenthesis and removing trailing function words, modals and adverbs, so a heading never ends mid-phrase (Algorithm 2).

Algorithm 2 Visual decision for one slide

```
if a table, section list or reference list is found then
    return it
end if
C ← {d(slide) : d ∈ D} \ {∅}; apply X
if C = ∅ then
    return key-point cards (lossless)
end if
return d*, its reason and the next three alternatives
```

VI. PSF and CLASS

A. Pedagogical Slide Fitness

PSF scores a slide s from three components in $[0,1]$.

Information efficiency. With $\mathcal{N}(s)$ the concepts introduced on s that did not appear on earlier slides and a corpus-free rarity weight $\mathrm{idf}(w) \in [0.4, 3.0]$ that grows with word length and is lower for common words,

$$U(s) = \sum_{w \in \mathcal{N}(s)} \mathrm{idf}(w), \qquad E(s) = \frac{U(s)}{U(s) + \kappa_e}\big(1 - \rho(s)\big), \tag{7}$$

where $\rho(s)$, the redundancy, is the mean Jaccard overlap of each bullet with the title or with the other bullets.

Cognitive load. With k distinct concepts, relational-cue density r, word count t, the largest visual element count v (table rows, flow nodes, cards) and a split-attention flag σ_s,

$$L(s) = \mathrm{sigm}\big(w_e \hat{e} + w_t \hat{t} + w_v \hat{v} + w_\sigma \sigma_s - \theta\big), \tag{8}$$

$$\hat{e} = \min\Big(1, \tfrac{k(1+r)}{e_{\mathrm{ref}}}\Big), \quad \hat{t} = \min\Big(1, \tfrac{t}{t_{\mathrm{ref}}}\Big), \quad \hat{v} = \min\Big(1, \tfrac{v}{v_{\mathrm{ref}}}\Big). \tag{9}$$

Multimedia coherence. $C(s)$ is the mean of four indicators from Mayer's principles: signalling (a real title and a take-away), coherence (share of sentences on the slide's topic), non-redundancy (a picture's description does not repeat the text) and spatial contiguity (the figure is on the same slide as its text).

Combination. The slide score is a Cobb–Douglas product, so very high load cannot be compensated by more information:

$$\mathrm{PSF}(s) = E(s)^{\alpha}\big(1 - L(s)\big)^{\beta} C(s)^{\gamma}, \qquad \alpha+\beta+\gamma = 1. \tag{10}$$

For a deck $S = (s_1, \ldots, s_n)$ the score is the geometric mean times a flow term that rewards moderate concept overlap o_i between consecutive slides:

$$\mathrm{PSF}(S) = \Big(\prod_{i=1}^{n} \mathrm{PSF}(s_i)\Big)^{1/n} \Phi, \qquad \Phi = \frac{1}{n-1} \sum_{i=1}^{n-1} \phi(o_i), \tag{11}$$

where $\phi(o) = 1$ for $o \in [0.18, 0.55]$, rises linearly from 0.6 below that band and falls towards 0.4 above it. Table III gives the default parameters.

TABLE III
PSF and CLASS parameters (defaults)

Parameter	Meaning	Value
α, β, γ	weights of E, $1-L$, C	0.40, 0.42, 0.18
κ_e	half-saturation of U	6
w_e, w_t, w_v, w_σ	load weights	1.10, 1.35, 0.85, 1.60
θ	load bias	2.30
$e_{\mathrm{ref}}, t_{\mathrm{ref}}, v_{\mathrm{ref}}$	load references	9, 16, 8
λ	cost of one more slide	0.35
μ	weight of $1 - E$	0.9
ν	overflow penalty	0.25

B. CLASS: load-optimal segmentation

Given the m points $b_1, \ldots, b_m$ of an overloaded slide, a hard cap of P points per slide and a soft target T, CLASS chooses break points $0 = j_0 < j_1 < \cdots < j_K = m$ that minimise

$$J = \sum_{k=1}^{K} \Big[L(G_k) + \mu\big(1 - E(G_k)\big) + \nu \max\big(0, |G_k| - T\big)^2 \Big] + \lambda K, \tag{12}$$

where $G_k = (b_{j_{k-1}+1}, \ldots, b_{j_k})$ and $|G_k| \le P$. Because the cost of a group depends only on its own points, the optimum follows from

$$F(i) = \min_{\max(0, i-P) \le p < i} \big[F(p) + \mathrm{cost}(b_{p+1..i}) + \lambda \big], \qquad F(0) = 0, \tag{13}$$

in $O(mP)$ time (Algorithm 3). Every point is placed exactly once, so segmentation never drops content.

Algorithm 3 CLASS

$F(0) \leftarrow 0$; $F(i) \leftarrow \infty$ for $i > 0$
for $i = 1$ to m **do**
  for $p = \max(0, i - P)$ to $i - 1$ **do**
    $c \leftarrow F(p) + \mathrm{cost}(b_{p+1..i}) + \lambda$
    if $c < F(i)$ **then**
      $F(i) \leftarrow c$; $cut(i) \leftarrow p$
    end if
  end for
end for
follow cut back from m to read off the groups

VII. Layout Without Broken Words

Lines are counted with an average advance ratio $\bar{\rho}$ (0.52 regular, 0.575 bold): a box W inches wide holds $\lfloor 72W/(\bar{\rho}s) \rfloor$ characters per line at size s. Single words are measured glyph by glyph,

$$w(u, s) = \kappa \, \frac{s}{72} \sum_{ch \in u} \rho(ch), \tag{14}$$

with per-character advances ρ from the font metrics and a safety factor $\kappa = 1.06$; words are cut into unbreakable pieces only after hyphens and slashes. The chosen size is

$$s^* = \max\{s \in \mathcal{G} : H(t, s, W) \le H\}, \qquad \mathcal{G} = \{s_{\max}, s_{\max} - 0.5, \ldots, s_{\min}\} \tag{15}$$

with $s_{\min} = 11\,\mathrm{pt}$ for body text; content that does not fit is moved to a continuation slide by CLASS instead of being shrunk further. Each paragraph is then scaled by

$$\sigma = \max\Big(0.62,\ \min\big(1,\ \min_{u} \tfrac{W}{w(u,s)}\big)\Big), \tag{16}$$

which guarantees that the longest word fits on one line. Wide visuals (processes, timelines, tables) keep the full slide width; side pictures are placed only on layouts with room for them.

A. Algorithms and equations kept whole

Lecture slides often hold pseudocode or a short derivation. Splitting such a slide into cards destroys loops and the order of a derivation, so a detector runs before all others. For the n distinct lines $\ell_1..\ell_n$ of a slide (duplicates and bare tick labels removed) let c be the number that start with a control word or contain an assignment (*for, while, repeat, if, return,* $\leftarrow$*,* :=) and e the number that are short equations. The slide becomes one ALGORITHM panel when

$$c \ge 3 \wedge \big(c \ge 0.5n \vee (\tau \wedge c \ge 0.34n)\big), \tag{17}$$

where τ is true if the title names an algorithm, and one EQUATIONS panel when $e \ge 3 \wedge e \ge 0.6n$. Indentation lost in export is recovered with a depth counter: $d_{i+1} = d_i + [\ell_i \text{ opens a block}]$ and $d_i \leftarrow d_i - 1$ when ℓ_i starts with *end, until, else*. Equations are parsed from their linear form ("$J(w,b) = (1/2m)\,\Sigma_{(i=1)}^{m}(h(x_i) - y_i)^2$") into a tree of fractions, powers, subscripts, roots and n-ary operators, which is written as Office Math in the PPTX, so PowerPoint shows a native, editable equation as Word does, and as MathML in the web deck; a Unicode form (`x^2` becomes x^2, `beta_0` becomes β_0) is the fallback for viewers without equation support.

VIII. Illustrations and Key Scheduling

Every topic slide receives a picture, in this order: the source's own figure, a diagram written by a language model, and an offline scene. The model is asked for one SVG in a 600×720 view box showing the mechanism of the slide's idea with the slide's own terms. The answer is *sanitised*: unsafe or unsupported elements (scripts, foreign objects, embedded images, filters, style sheets) are removed, inline styles become attributes, any external reference rejects the diagram, and it must contain at least five shapes and render. Labels that would cross the edge are scaled with the width estimate of Section VII. An accepted diagram is cached by a hash of the slide text, embedded as live SVG in the web deck, rasterised for PowerPoint and published as a page to which the PowerPoint picture links.

Only free-tier keys are used, so a model is called only when the offline result is weak: Tesseract's text fails a dictionary readability test, a title is a bare label or the slide is under 60 words, or the slide needs a drawn mechanism, plot, anatomy or comparison that an offline scene cannot show; the figure kind is chosen per slide before any request. On eight real inputs (599 output slides) this kept 37% of picture reads and 30% of diagram requests offline, and every deck completed after the daily free quota ran out.

Requests are spread over a pool K of keys. Each key is paced on its own,

$$t_{\mathrm{next}}(k) = \max\big(t_{\mathrm{now}},\ t_{\mathrm{last}}(k) + \Delta\big), \qquad \Delta = 4.2\,\mathrm{s}, \tag{18}$$

two requests are kept in flight per key, and a key that returns an authentication or quota error is skipped for $T_d = 15$ minutes. With $|K|$ keys the sustained rate is $|K|/\Delta$ requests per second, about 1.2 for five keys (Algorithm 4).

Algorithm 4 Illustration with fallbacks

for each topic slide s, in parallel with $2|K|$ workers **do**
  if s shows a source figure **then**
    use it; **continue**
  end if
  if s has a wide visual **then**
    header picture (offline); **continue**
  end if
  if $\mathrm{hash}(s)$ is cached **then**
    use the cached diagram; **continue**
  end if
  for each live key k in round-robin order **do**
    wait for the pace of k; $a \leftarrow \textsc{Ask}(k, s)$
    if authentication or quota error **then**
      skip k for T_d; **continue**
    end if
    $g \leftarrow \textsc{Sanitise}(a)$; **break**
  end for
  attach g (SVG, PNG, page) or an offline scene
end for

IX. Evaluation

A. Setup

We used three gold sets of slide texts labelled with their acceptable visual families (development 38, held-out 32 and blind 31; the blind set was written by a member who had not seen the rules) and six real course decks: four slide PDFs of 36, 44, 95 and 216 pages and two PowerPoint files of 28 and 63 slides, named A–F in the tables. All runs used a laptop CPU and no API key unless stated.

B. Visual decisions

Table IV reports strict accuracy: the family is acceptable, is drawn as a real visual and receives data. Each set had a single miss, in each case a slide on which two families were defensible. The median decision time was 0.48 ms per slide (95th percentile 0.82 ms). Fig. 2 shows the families chosen over the six rebuilt decks; figures dominate because the sources contain many picture slides, followed by key-point cards, outlines, prose and comparison tables.

TABLE IV
Visual decision accuracy on the gold sets

Set	Slides	Strict acc.	Real visual	Families
Development	38	97.4%	100%	24
Held-out	32	96.9%	100%	21
Blind	31	96.8%	100%	21

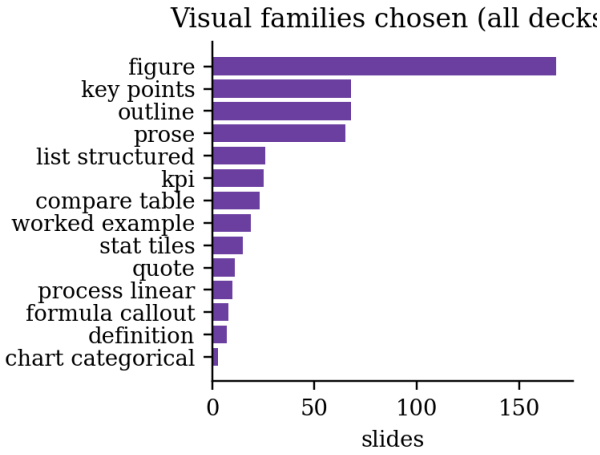


Fig. 2. Visual families chosen over the six rebuilt decks.

C. Fidelity, layout and speed

Table V reports the rebuilt decks. Fact retention is

$$R = \frac{|F_{\mathrm{src}} \cap F_{\mathrm{deck}}|}{|F_{\mathrm{src}}|}, \tag{19}$$

where F is the set of numbers with units, years, percentages and capitalised terms. No slide was flagged for overflow or for a broken word; every flag marked a slide that is sparse because its source held a single short line. Run time grows roughly linearly with length (Fig. 3); the PowerPoint decks take longer per page because they are first rendered to PDF and contain many full-page figures.

TABLE V
Rebuilt decks without an API key

Deck	Pages	Slides	Time (s)	R	Flagged
A (PPTX)	28	33	11.7	100%	6%
B (PPTX)	63	69	44.0	100%	7%
C (PDF)	36	47	9.8	100%	0%
D (PDF)	44	53	14.8	98%	4%
E (PDF)	95	100	21.2	99%	3%
F (PDF)	216	225	34.4	100%	3%

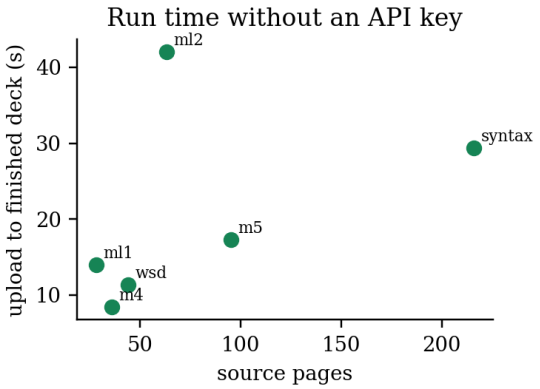


Fig. 3. Time from upload to finished files against source length (no API key).

D. Structural change

Table VI and Fig. 4 compare each source deck with its rebuilt deck. Untitled slides fell from 83 to 0 across the five decks, and the rebuilt decks show a picture on most slides and a structured visual on most slides, where the sources had none. The PowerPoint route matters: on deck B the earlier shape-by-shape parser produced 23 slides from 63 and titles such as “Slide 9”, while the PDF route produced 68 slides covering every source slide.

TABLE VI
Source deck against rebuilt deck (with keys)

Deck	Untitled (source)	Pictures src → new	Visual slides	Quiz slides	Lab links	Linked illustr.
A	14	22 → 29	22	5	2	12
B	36	49 → 64	45	4	8	23
C	0	13 → 43	43	6	3	37
D	1	7 → 46	52	7	10	40
E	32	49 → 96	89	6	13	78

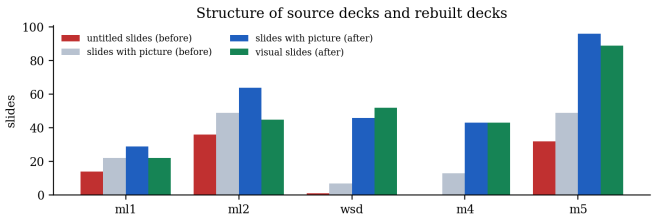


Fig. 4. Untitled slides and pictures in the source decks, and pictures and visual slides in the rebuilt decks.

E. PSF and CLASS

Table VII and Fig. 5 give the PSF components of each source deck read as-is and of its rebuilt deck. Coherence C rose on five of the six decks, because every slide gains a real title, a takeaway and a picture next to its text. Mean PSF rose clearly on the two long PDF decks (E: $0.37 \rightarrow 0.49$; F: $0.15 \rightarrow 0.24$) and was unchanged on C, but fell on A, B and D. The components show why: on those decks the rebuilt slides carry more words, since explanations, examples and key terms are added to thin slides; this raises the estimated load L, and because the added sentences repeat concepts already introduced, it lowers E. PSF therefore penalises exactly the enrichment we added for readability, a calibration question we return to in Section X.

TABLE VII
PSF COMPONENTS: SOURCE DECK / REBUILT DECK

Deck	E	L	C	mean PSF
A	0.35 / 0.30	0.31 / 0.32	0.78 / 0.84	0.34 / 0.29
B	0.24 / 0.23	0.25 / 0.30	0.77 / 0.84	0.28 / 0.26
C	0.46 / 0.44	0.39 / 0.40	0.80 / 0.77	0.40 / 0.40
D	0.56 / 0.51	0.46 / 0.44	0.76 / 0.78	0.52 / 0.46
E	0.40 / 0.54	0.33 / 0.44	0.77 / 0.78	0.37 / 0.49
F	0.14 / 0.25	0.20 / 0.31	0.76 / 0.83	0.15 / 0.24

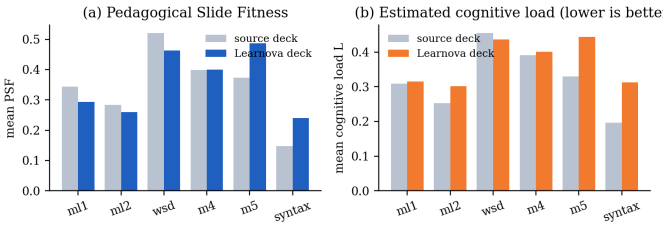


Fig. 5. (a) Mean PSF and (b) mean estimated cognitive load of each source deck (light) and its rebuilt deck (dark). PSF rises on the two long PDFs (E, F) and falls slightly on the short decks, where added explanations raise the load.

For CLASS we took every source slide with more than six points (75 sections) and split it with a cap of five points per slide, once with CLASS and once evenly. Table VIII and Fig. 6 show the result. Both methods used 179 slides in total; CLASS had a lower total cost on 28 sections (37%) and an equal cost on the rest. It never did worse, as expected of an exact optimiser of the same objective.

TABLE VIII
CLASS AGAINST EVEN SPLITTING (75 OVERLOADED SECTIONS)

Measure	Even	CLASS
Mean total cost J	2.799	2.780
Sections with lower cost	–	28 (37%)
Sections with higher cost	–	0
Slides produced	179	179

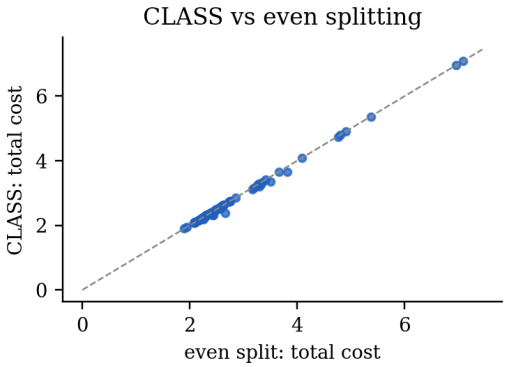


Fig. 6. Total cost per overloaded section, CLASS against even splitting. Points on or below the diagonal favour CLASS.

F. Illustrations

Without a key every topic slide receives an offline scene. With the five-key pool, ten diagrams were written and accepted in 10.1 s in parallel. Before sanitisation replaced outright rejection, only 71 of 206 model answers (34.5%) had been accepted, most of them failing on inline styles and gradients; after the change, all ten diagrams of a live check passed.

X. Discussion and Limitations

The rule-based design costs some coverage, but every decision can be read, tested and corrected, it takes under a millisecond, and it works without a network. The PSF results show that a metric built from theory does not automatically agree with every design choice: the explanations we add to thin slides help a reader but raise the estimated load. PSF's weights are literature priors; calibrating them against student ratings, and a classroom study of learning outcomes, are our next steps. OCR of photographed equations remains the weakest part of the reader, which is why such pictures are kept rather than transcribed.

XI. Conclusion

Learnova rebuilds the decks teachers already use rather than generating new ones. Its methods—structure recovery rules, an explainable decision engine, the PSF metric, CLASS segmentation, glyph-measured layout and sanitised AI diagrams with offline fallbacks—kept 98–100% of the source facts on real decks, removed every untitled slide and never broke a word, while running in seconds on ordinary hardware.

ACKNOWLEDGMENT

The authors thank the faculty of the Department of Artificial Intelligence and Data Science, DJSCE, for their guidance and for the lecture material used in the evaluation.

REFERENCES

[1] Gamma Tech, "Gamma: AI-powered presentations," https://gamma.app, 2026, accessed Oct. 2026.

[2] Microsoft, "Copilot in PowerPoint," https://support.microsoft.com/copilot-powerpoint, 2026, accessed Oct. 2026.

[3] Google, "Gemini in Google Slides," https://workspace.google.com/products/slides/, 2026, accessed Oct. 2026.

[4] Canva, "Magic design for presentations," https://www.canva.com/magic-design/, 2026, accessed Oct. 2026.

[5] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, "On faithfulness and factuality in abstractive summarization," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 1906–1919.

[6] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[7] Y. Hu and X. Wan, "PPSGen: Learning-based presentation slides generation for academic papers," *IEEE Transactions on Knowledge and Data Engineering*, vol. 27, no. 4, pp. 1085–1097, 2015.

[8] T.-J. Fu, W. Y. Wang, D. McDuff, and Y. Song, "DOC2PPT: Automatic presentation slides generation from scientific documents," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 1, 2022, pp. 634–642.

[9] E. Sun, Y. Hou, D. Wang, Y. Zhang, and N. X. R. Wang, "D2S: Document-to-slide generation via query-based text summarization," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics*, 2021, pp. 1405–1418.

[10] J. Mackinlay, "Automating the design of graphical presentations of relational information," *ACM Transactions on Graphics*, vol. 5, no. 2, pp. 110–141, 1986.

[11] D. Moritz, C. Wang, G. L. Nelson, H. Lin, A. M. Smith, B. Howe, and J. Heer, "Formalizing visualization design knowledge as constraints: Actionable and extensible models in Draco," *IEEE Transactions on Visualization and Computer Graphics*, vol. 25, no. 1, pp. 438–448, 2019.

[12] M. Alley, M. Schreiber, K. Ramsdell, and J. Muffo, "How the design of headlines in presentation slides affects audience retention," *Technical Communication*, vol. 53, no. 2, pp. 225–234, 2006.

[13] J. K. Garner and M. P. Alley, "How the design of presentation slides affects audience comprehension: A case for the assertion–evidence approach," *International Journal of Engineering Education*, vol. 29, no. 6, pp. 1564–1579, 2013.

[14] R. E. Mayer, *Multimedia Learning*, 2nd ed. Cambridge: Cambridge University Press, 2009.

[15] J. Sweller, "Cognitive load during problem solving: Effects on learning," *Cognitive Science*, vol. 12, no. 2, pp. 257–285, 1988.

[16] N. Cowan, "The magical number 4 in short-term memory: A reconsideration of mental storage capacity," *Behavioral and Brain Sciences*, vol. 24, no. 1, pp. 87–114, 2001.

[17] D. E. Knuth and M. F. Plass, "Breaking paragraphs into lines," *Software: Practice and Experience*, vol. 11, no. 11, pp. 1119–1184, 1981.

[18] R. Smith, "An overview of the Tesseract OCR engine," in *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.